C++ Programming Textbook Notes

Module: Default Arguments in C++ | Flexibility, Prototype Rules & Memory Mechanics

Author: Gaurav Kandpal **Platform:** CS-Simplified **Level:** Beginner to Advanced

1. DEFAULT ARGUMENTS IN C++

Conceptual Explanation

In standard function calls, the number of actual arguments passed at the call site must match the exact number of formal parameters declared in the function header. If a function is declared to accept three arguments (e.g., `int add(int, int, int)`), invoking it with only two arguments (e.g., `add(a, b)`) causes a compile-time error: *"too few arguments to function call"*.

To solve this and provide API flexibility, C++ introduces **Default Arguments**.

A **Default Argument** is a fallback value assigned to a formal parameter in the function declaration. If the caller does not supply an actual argument for that parameter during the function call, the compiler automatically substitutes the predefined default value.

Why Use Default Arguments?

- **Function Call Flexibility:** Enables calling the same function using different numbers of arguments (e.g., `add(a, b)` and `add(a, b, c)`).
- **Reduces Code Duplication:** Eliminates the need to write multiple overloaded functions that simply forward default values.
- **Backward Compatibility:** Allows adding new parameters to existing functions without breaking existing call sites in legacy code.

How Default Arguments Work in Memory

Default arguments do **not** create runtime overhead or dynamic branching logic. The substitution happens entirely at **compile time**:

1. When the compiler encounters a function call missing trailing arguments, it inspects the function prototype.
2. The compiler inserts the constant default value into the generated assembly code as if the programmer had written it explicitly.
3. During runtime execution, the default value is pushed onto the function's **Stack Frame (Activation Record)** alongside the other arguments.

```
[ COMPILER ARGUMENT SUBSTITUTION & STACK MEMORY LAYOUT ]

1. Function Declaration:
   int add(int x, int y, int z = 0);

2. Call Site 1: add(5, 6);
   Compiler Translation ──► Pushes 5 for 'x', 6 for 'y', and 0 (default) for 'z'
   Stack Frame in RAM:
   ┌──────────────────┬──────────────────┬──────────────────┐
   | x = 5 (4 Bytes)  | y = 6 (4 Bytes)  | z = 0 (4 Bytes)  | ──► z takes default value!
   └──────────────────┴──────────────────┴──────────────────┘

3. Call Site 2: add(5, 6, 7);
   Compiler Translation ──► Pushes 5 for 'x', 6 for 'y', and 7 (passed) for 'z'
   Stack Frame in RAM:
   ┌──────────────────┬──────────────────┬──────────────────┐
   | x = 5 (4 Bytes)  | y = 6 (4 Bytes)  | z = 7 (4 Bytes)  | ──► z overridden by passed argument!
   └──────────────────┴──────────────────┴──────────────────┘
```

2. SYNTAX & THE 3 GOLDEN RULES

Syntax Declaration

```
return_type function_name(type1 param1, type2 param2 = default_value);
```

Rule 1: Right-to-Left Trailing Order (The Cardinal Constraint)

Default arguments must always be assigned from **right to left** in the parameter list. Once a parameter is given a default value, **all subsequent parameters to its right must also have default values**.

```
[ VALIDITY MATRIX: RIGHT-TO-LEFT RULE ]

✓ VALID:   int add(int x, int y, int z = 0);          // Rightmost argument is default
✓ VALID:   int add(int x, int y = 0, int z = 0);      // Trailing 2 arguments are default
✓ VALID:   int add(int x = 0, int y = 0, int z = 0);  // All arguments are default

✗ INVALID: int add(int x = 0, int y, int z);          // ERROR: Non-default parameters follow default!
✗ INVALID: int add(int x, int y = 0, int z);          // ERROR: 'z' has no default after 'y'!
```

Rule 2: Declaration vs. Definition (Specify Only Once)

Default argument values should be specified in the **function declaration (prototype)**, *not* in both the declaration and definition. Specifying default values in both locations causes a compilation error (redefinition of default argument).

Rule 3: Positional Argument Mapping

During invocation, arguments are matched strictly by position from **left to right**. You cannot skip an intermediate parameter to provide a value for a later parameter.

3. DEFAULT ARGUMENTS VS. FUNCTION OVERLOADING

Feature	Default Arguments	Function Overloading
Function Bodies	Single function definition handles multiple call varieties.	Requires writing distinct function definitions for each parameter combination.
Compile-Time Mechanism	Compiler fills in missing default constants at call sites.	Compiler selects appropriate function based on name mangling and signature.
Code Maintenance	High. Changes to algorithm logic are made in one place.	Lower. Logic must be maintained across multiple overloaded bodies.
Flexibility	Applies when missing arguments have logical default constants.	Applies when functions have entirely different algorithms or types.

4. PRACTICAL CODE DEMONSTRATIONS

Example 1: Basic Default Argument Addition Program

```
// Demonstrating Default Arguments in C++
#include <iostream>
using namespace std;

// Function Declaration: 'z' has a default value of 0
int add(int x, int y, int z = 0);

int main() {
    int a, b, c;

    // Case 1: Calling with 2 arguments (z defaults to 0)
    cout << "Enter two numbers: ";
    cin >> a >> b;
    cout << "Sum of 2 numbers = " << add(a, b) << endl; // Evaluates as add(a, b, 0)

    // Case 2: Calling with 3 arguments (z takes passed value 'c')
    cout << "
Enter three numbers: ";
    cin >> a >> b >> c;
    cout << "Sum of 3 numbers = " << add(a, b, c) << endl; // Evaluates with passed 'c'

    return 0;
}

// Function Definition (Default values are NOT repeated here)
int add(int x, int y, int z) {
    return (x + y + z);
}
```

Example 2: Multiple Default Parameters

```cpp
// Function calculating simple interest with multiple defaults
#include <iostream>
using namespace std;

// Rate defaults to 5.0%, Time defaults to 1 year
double calculateInterest(double principal, double rate = 5.0, int time = 1);

int main() {
    // Call 1: Uses default rate (5.0) and default time (1)
    cout << "Interest 1: $" << calculateInterest(1000.0) << endl;

    // Call 2: Overrides rate (7.5), uses default time (1)
    cout << "Interest 2: $" << calculateInterest(1000.0, 7.5) << endl;

    // Call 3: Overrides both rate (8.0) and time (3)
    cout << "Interest 3: $" << calculateInterest(1000.0, 8.0, 3) << endl;

    return 0;
}

double calculateInterest(double principal, double rate, int time) {
    return (principal * rate * time) / 100.0;
}
```

Common Gotcha: Ambiguity with Function Overloading

Be careful when combining default arguments with function overloading! For example:

`void print(int x);` and `void print(int x, int y = 10);`

Calling `print(5);` causes a compiler error: *"call to 'print' is ambiguous"* because both functions match!

Best Practice: Declaration Location

Always define default argument values in the public header file declaration (prototype) so callers including the header know the available default values.